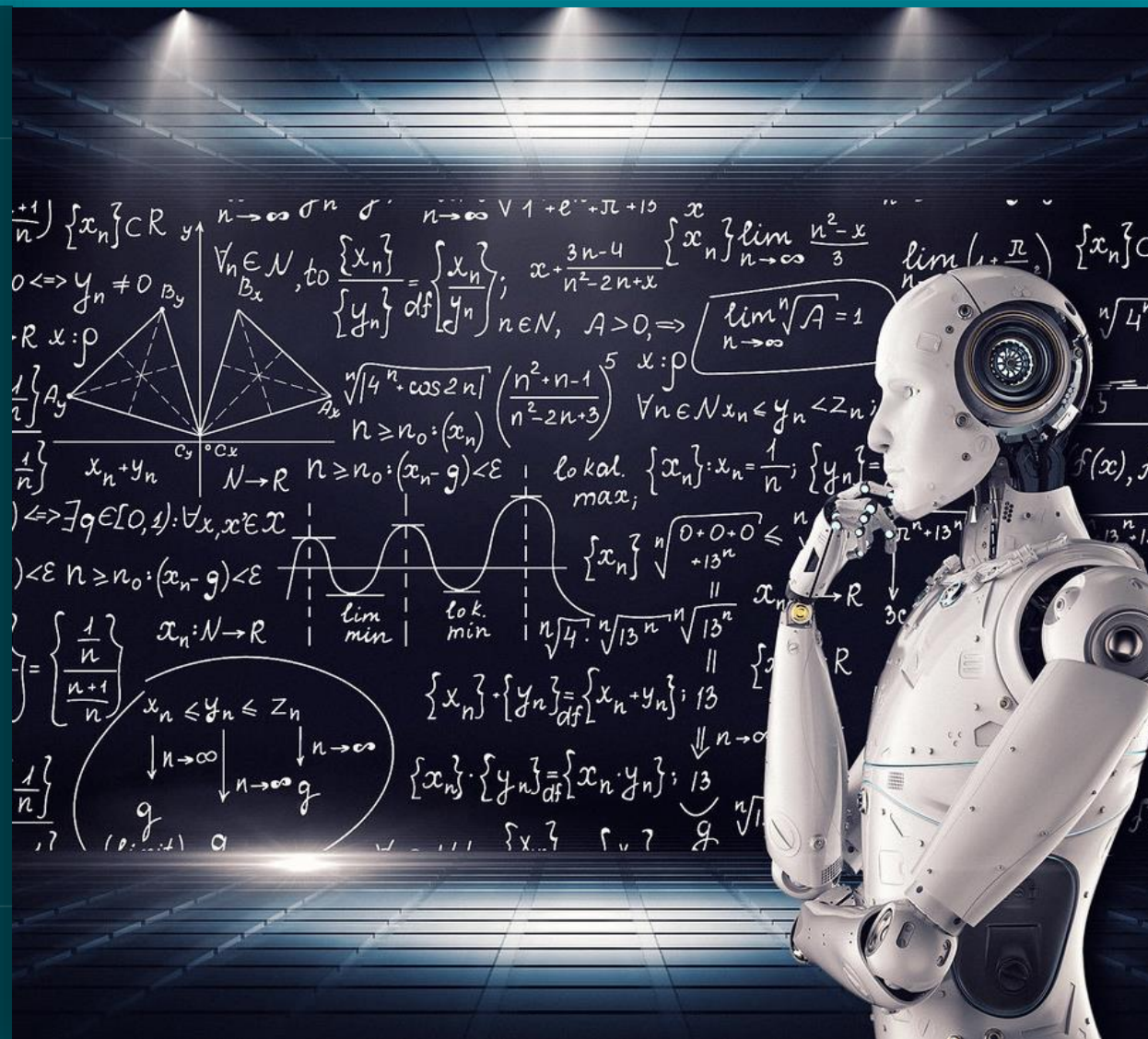


Classification



What is classification?

- Assigning observations to one of a **finite set of classes**

Binary classification

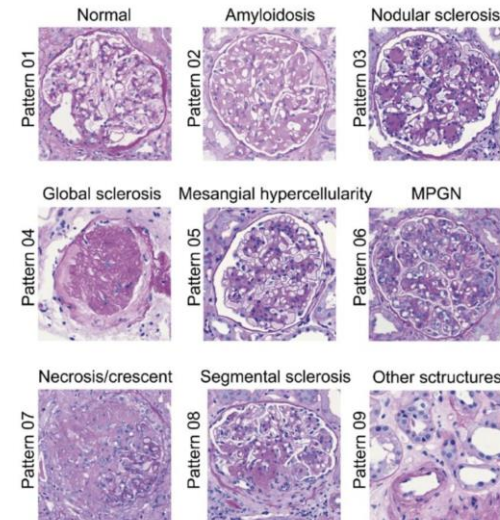


Sick



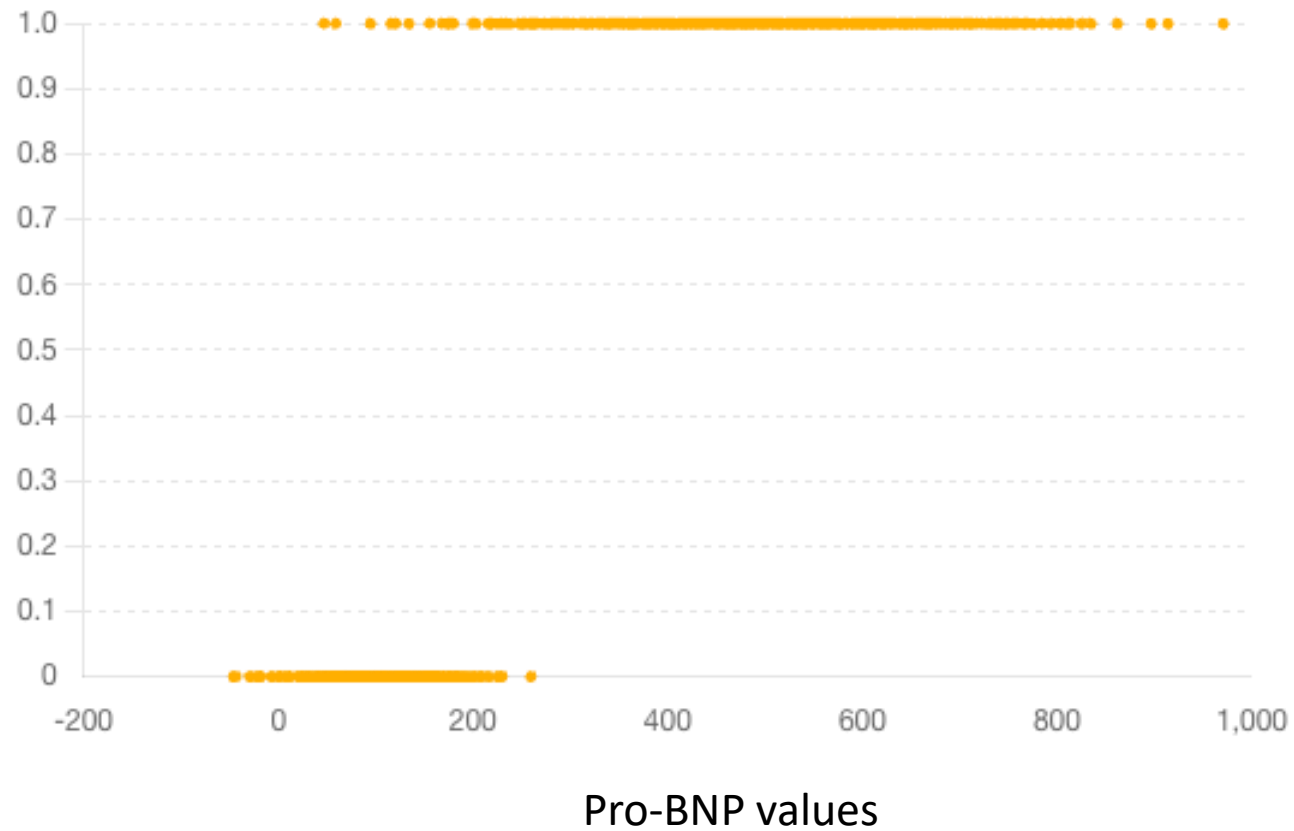
Healthy

Multiclass classification



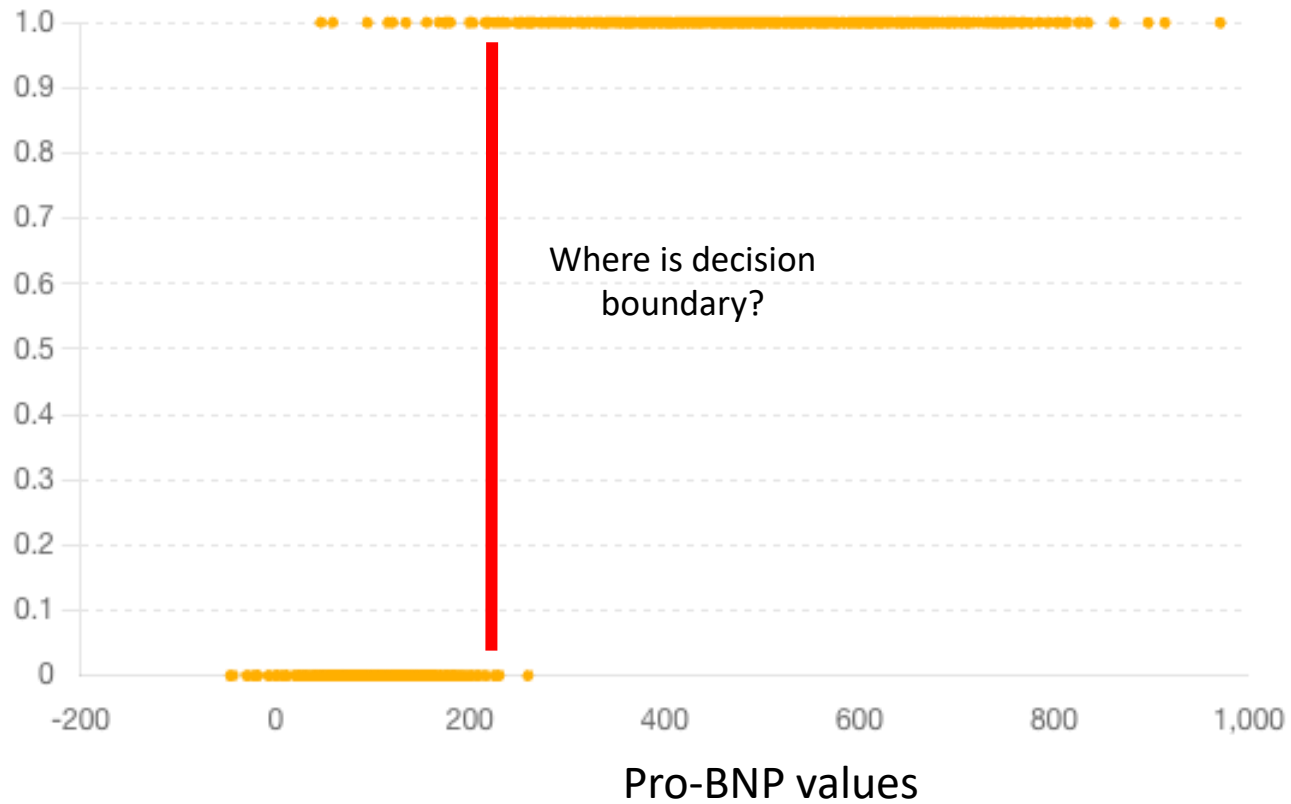
How to classify?

Heart Failure 1 = yes
Heart Failure 0 = no



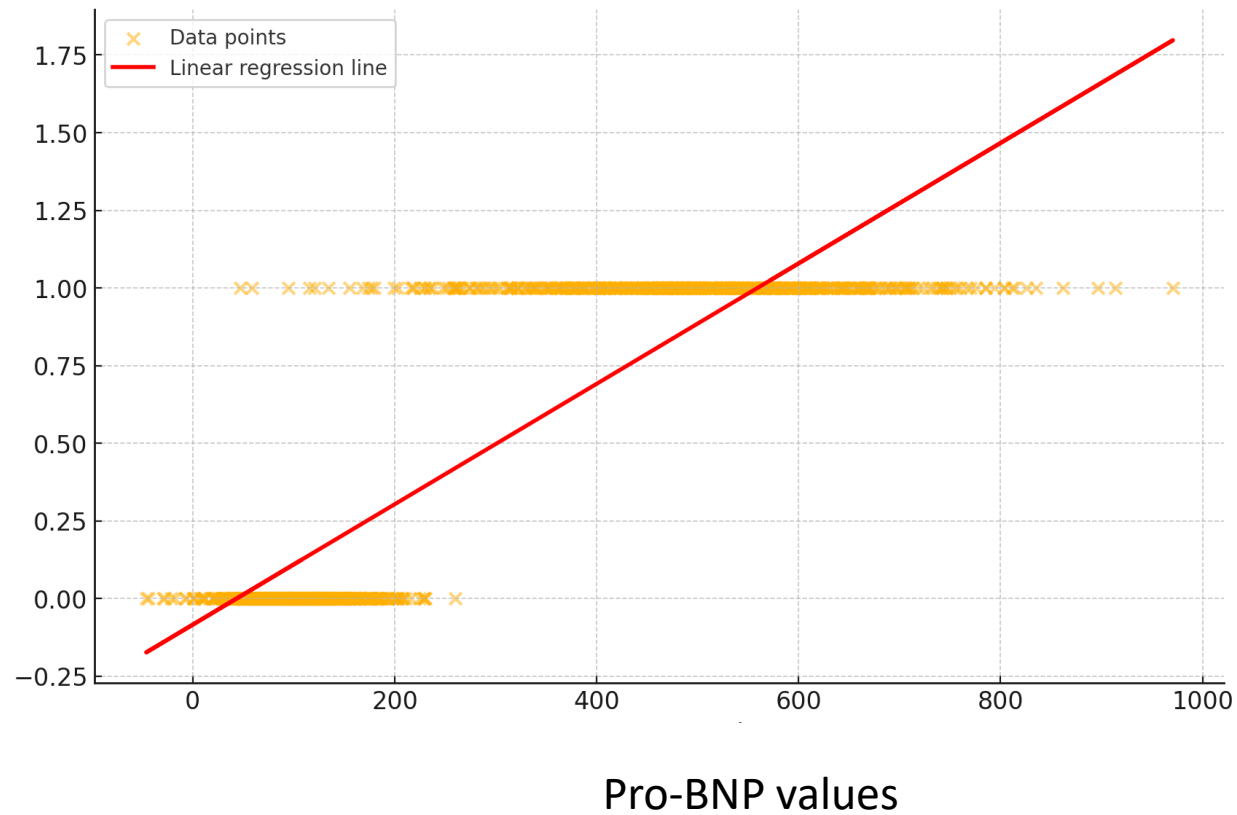
How to classify?

Heart Failure 1 = yes
Heart Failure 0 = no



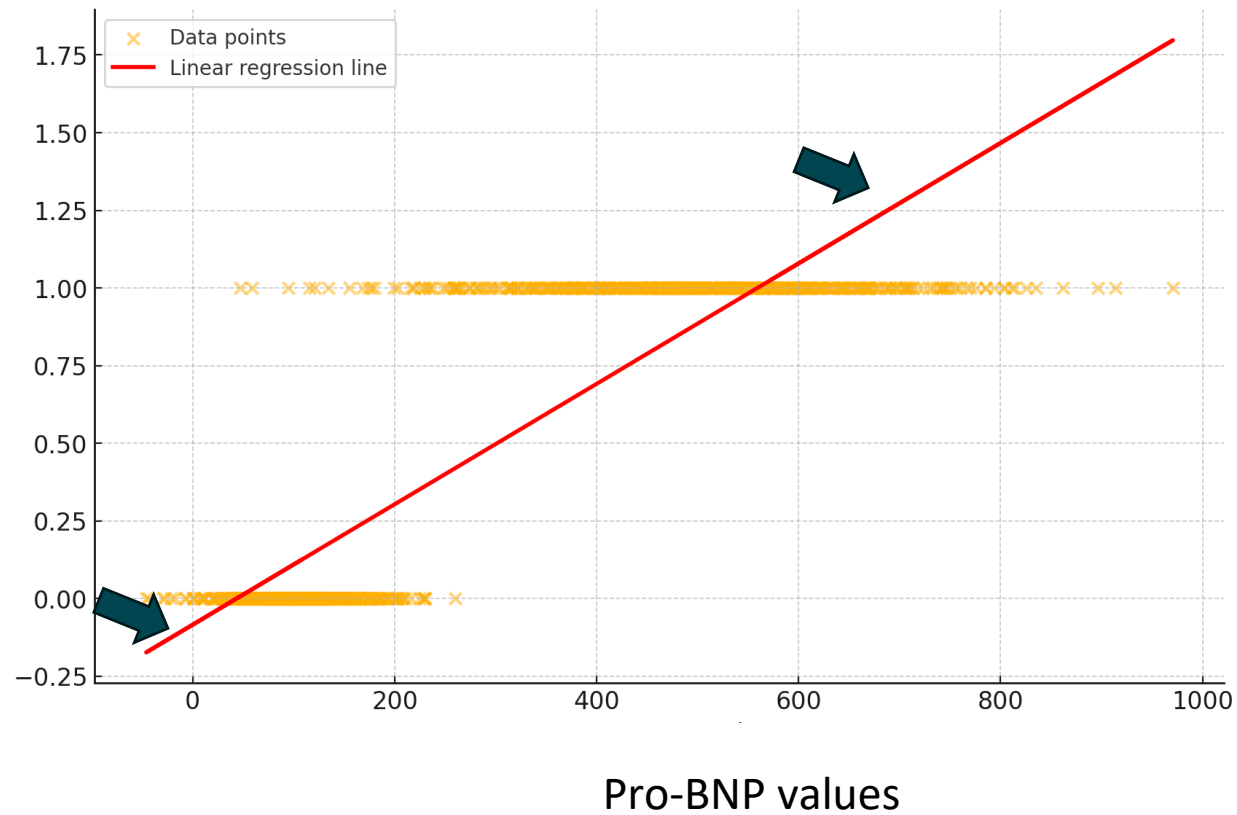
Would a linear regression help?

Heart Failure 1 = yes
Heart Failure 0 = no



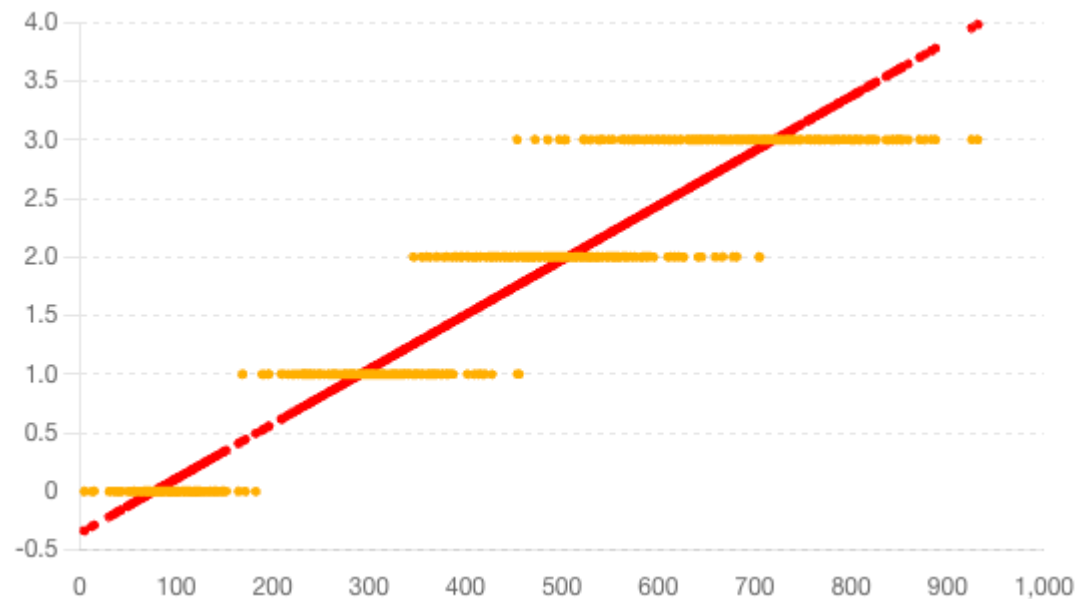
Would a linear regression help?

Heart Failure 1 = yes
Heart Failure 0 = no



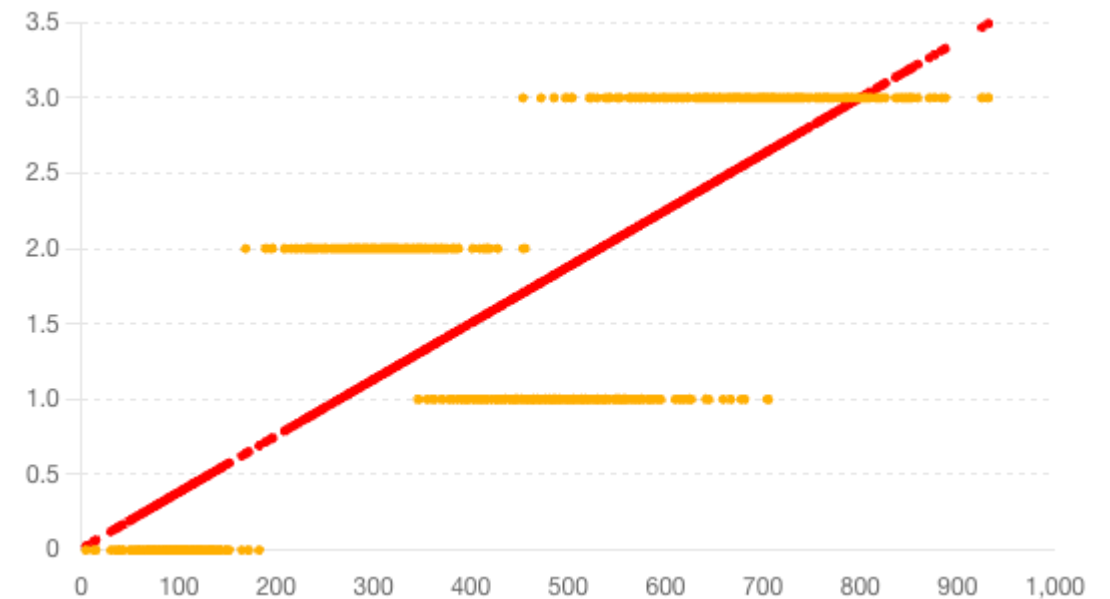
Would a linear regression help?

(0: No HF, 1: Preserved EF, 2: Mid Range EF, 3: Reduced EF)



- Coefficient: 0.00466
- Intercept: -0.35743

(0: No HF, 2: Preserved EF, 1: Mid Range EF, 3: Reduced EF)



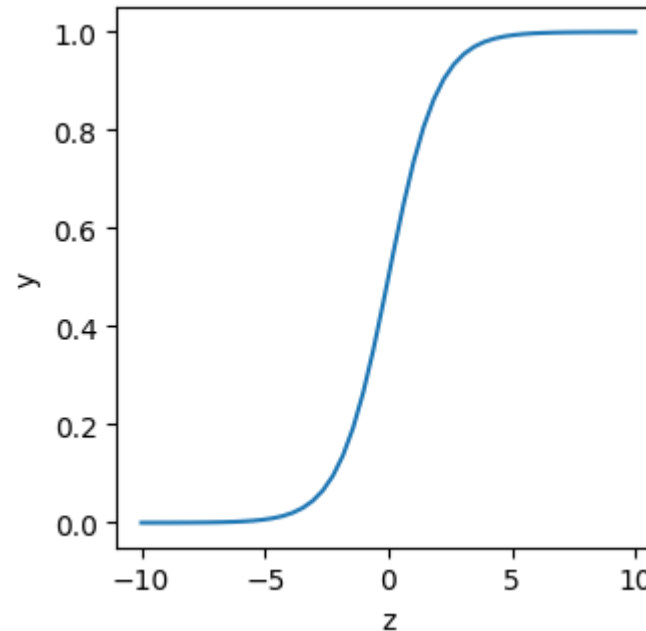
- Coefficient: 0.00374
- Intercept: 0.00989

Logistic regression

- Sigmoid function

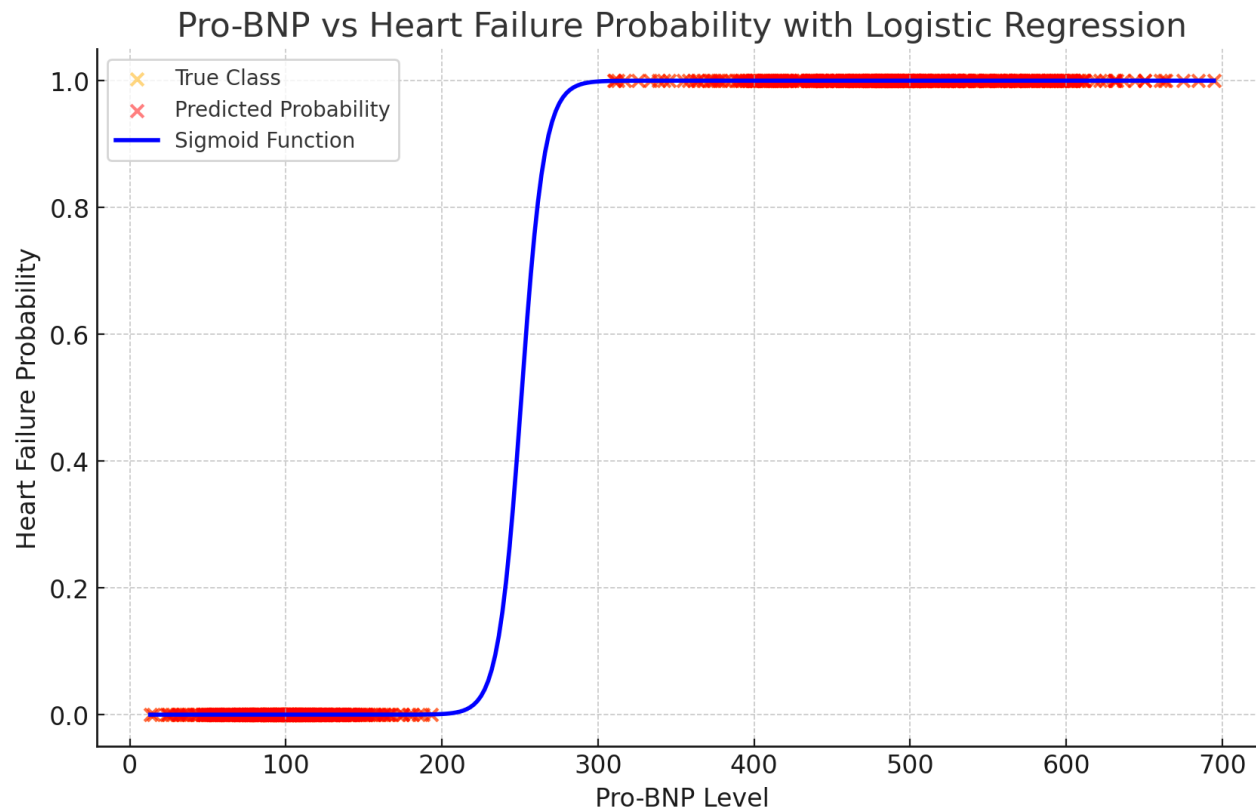
$$y = \frac{1}{1 + e^{-z}}$$

$$z = \theta_0 + \theta_1 X$$



We are still dealing with limited parameters!

Logistic regression



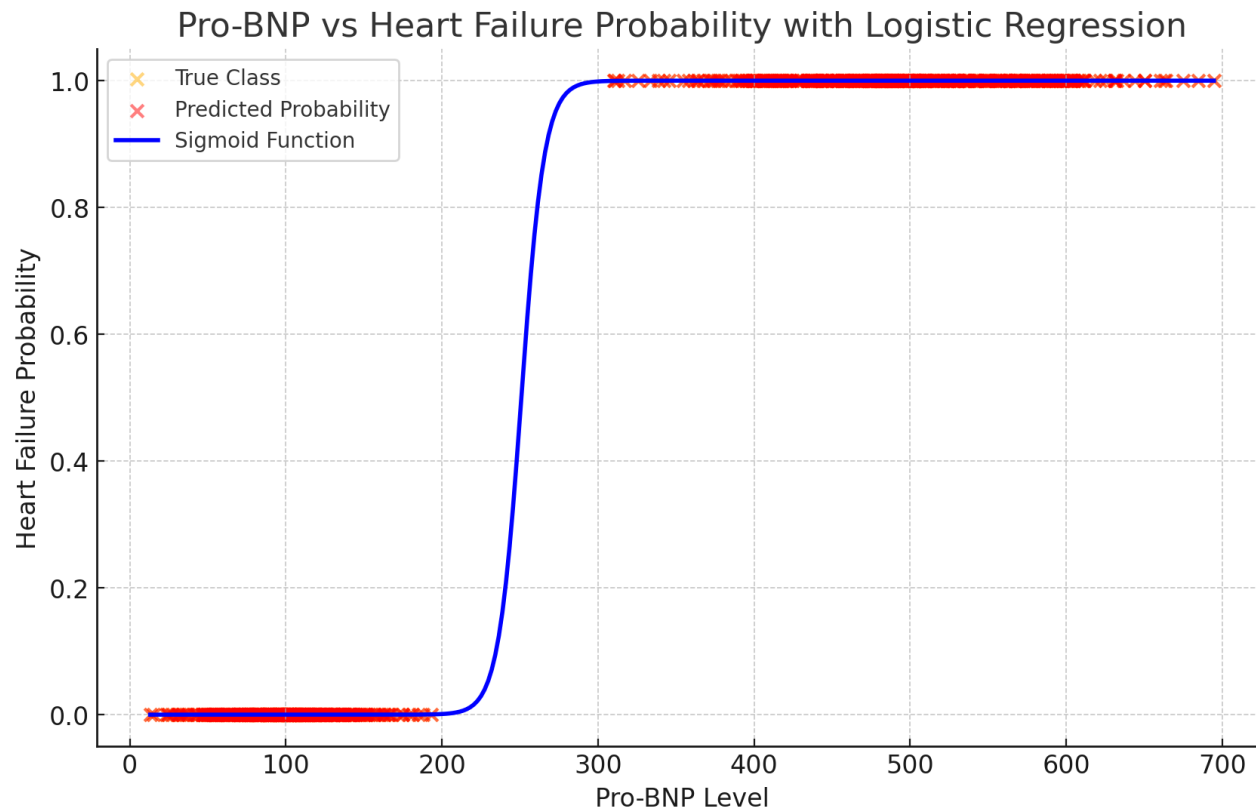
$$P(y = \text{HF} \mid x = \text{pro-BNP})$$

$$P(y = 1|X) = \frac{1}{1+e^{-(B_0+B_1X)}}$$

- B_0 (intercept): -33.9817
- B_1 (coefficient for pro-BNP): 0.1355

Now we have a probability between 0 and 1!

Logistic regression



$$P(y = \text{HF} \mid x = \text{pro-BNP})$$

$$P(y = 1|X) = \frac{1}{1+e^{-(\beta_0+\beta_1 X)}}$$



Odds ratio

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0 + \beta_1 X}$$



$$\log \left(\frac{p(X)}{1 - p(X)} \right) = \beta_0 + \beta_1 X$$

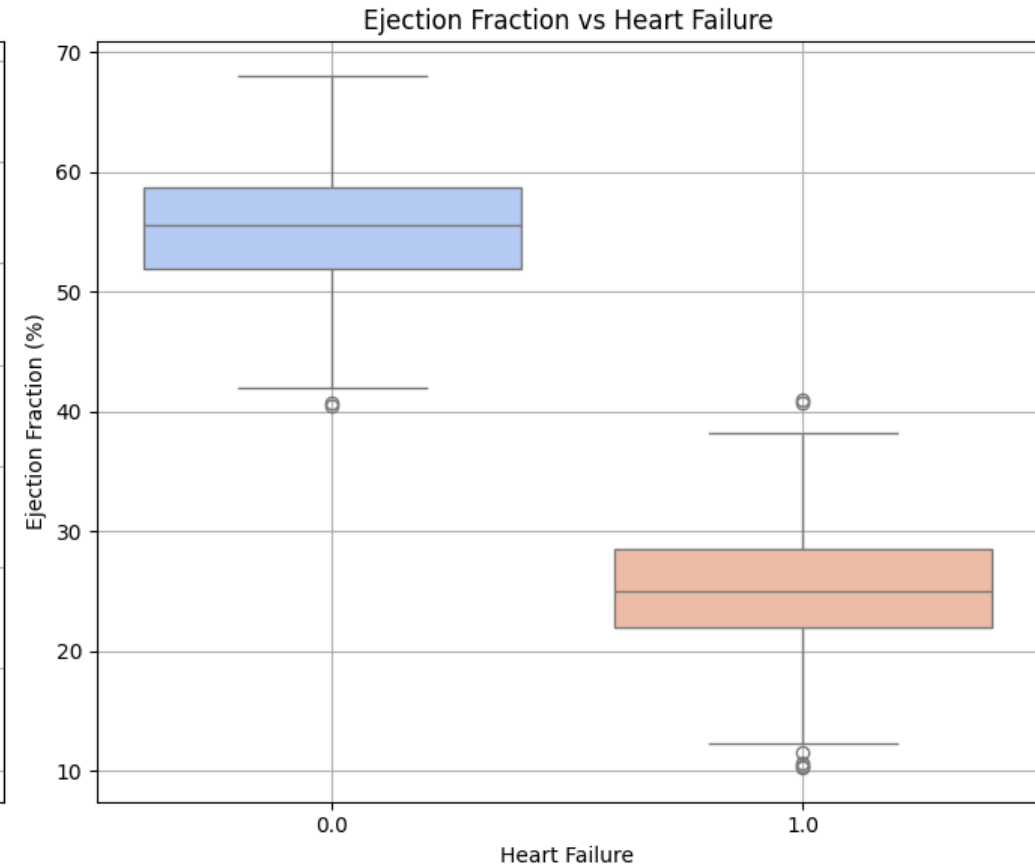
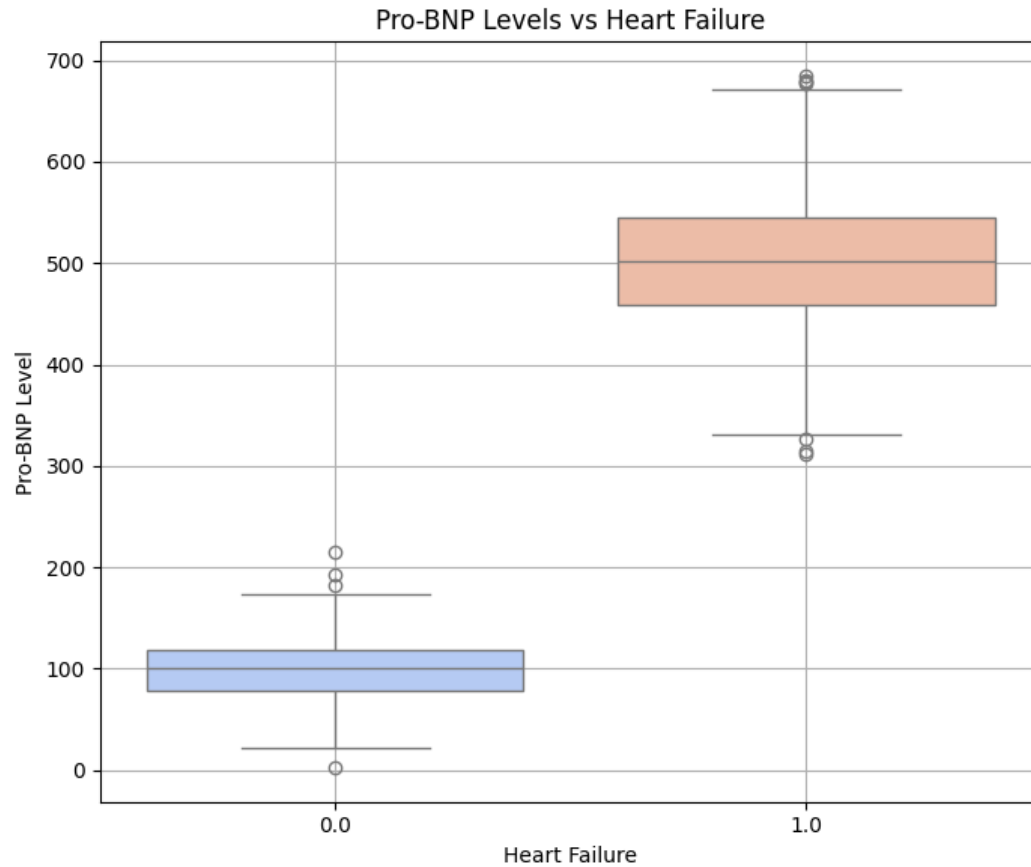
Multiple logistic regression

- Using multiple predictors

$$\hat{y} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k)}}$$

$$\log \left(\frac{\hat{y}}{1 - \hat{y}} \right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k$$

Multiple logistic regression



Multiple logistic regression

Log Odds Formula:

$$\log \left(\frac{\hat{y}}{1 - \hat{y}} \right) = \beta_0 + \beta_1 \cdot \text{Pro_BNP_Level} + \beta_2 \cdot \text{Ejection_Fraction}$$

Where:

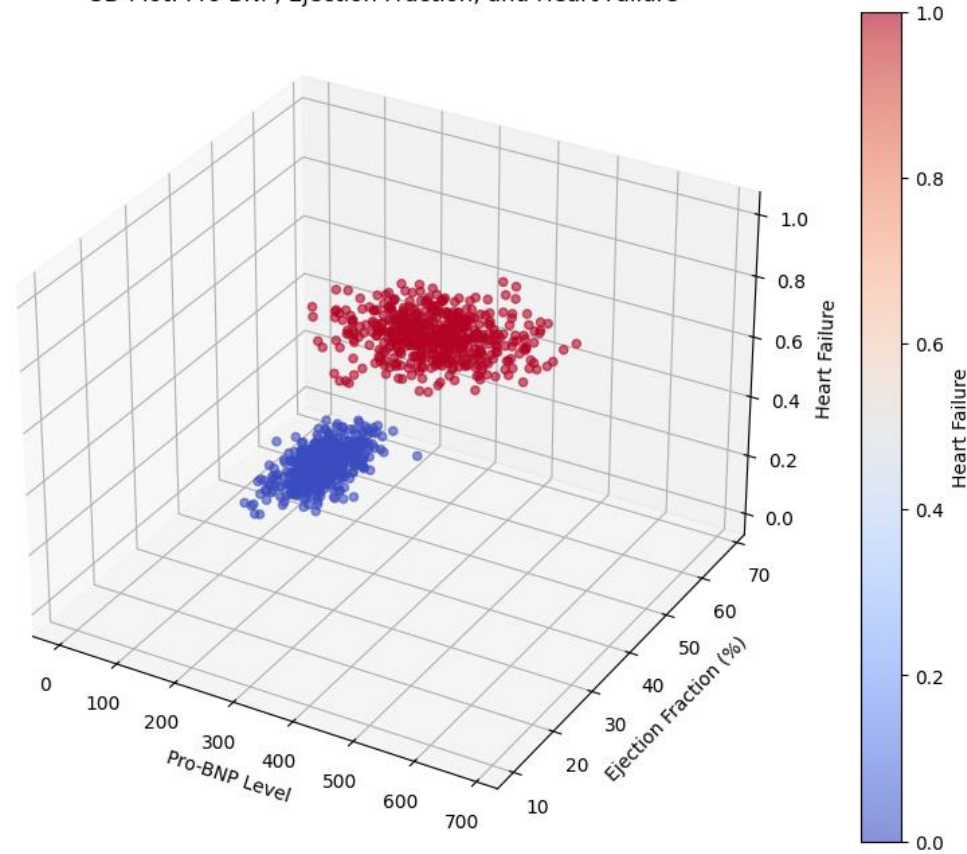
- $\hat{y}$ is the predicted probability of heart failure.
- β_0 is the intercept term.
- β_1 is the coefficient for Pro-BNP Level.
- β_2 is the coefficient for Ejection Fraction.

Probability Prediction Formula:

$$\hat{y} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \cdot \text{Pro_BNP_Level} + \beta_2 \cdot \text{Ejection_Fraction})}}$$

Multiple logistic regression

3D Plot: Pro-BNP, Ejection Fraction, and Heart Failure



Estimating regression coefficients or fitting the model

- Log-likelihood cost function

$$\log L(\beta) = \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

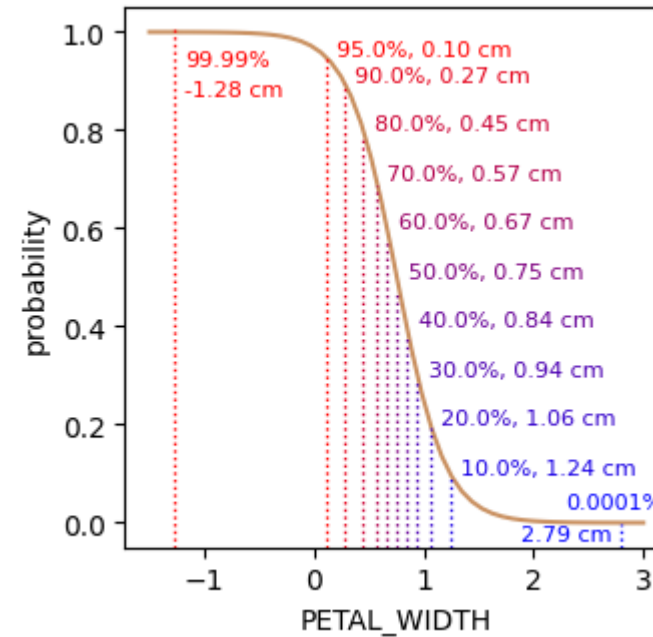
Goal: maximize the log-likelihood cost function (ie. Minimizing the absolute value)

From probabilities to classification

- Different metrics for assessing accuracy
- How are observations "correctly classified"?

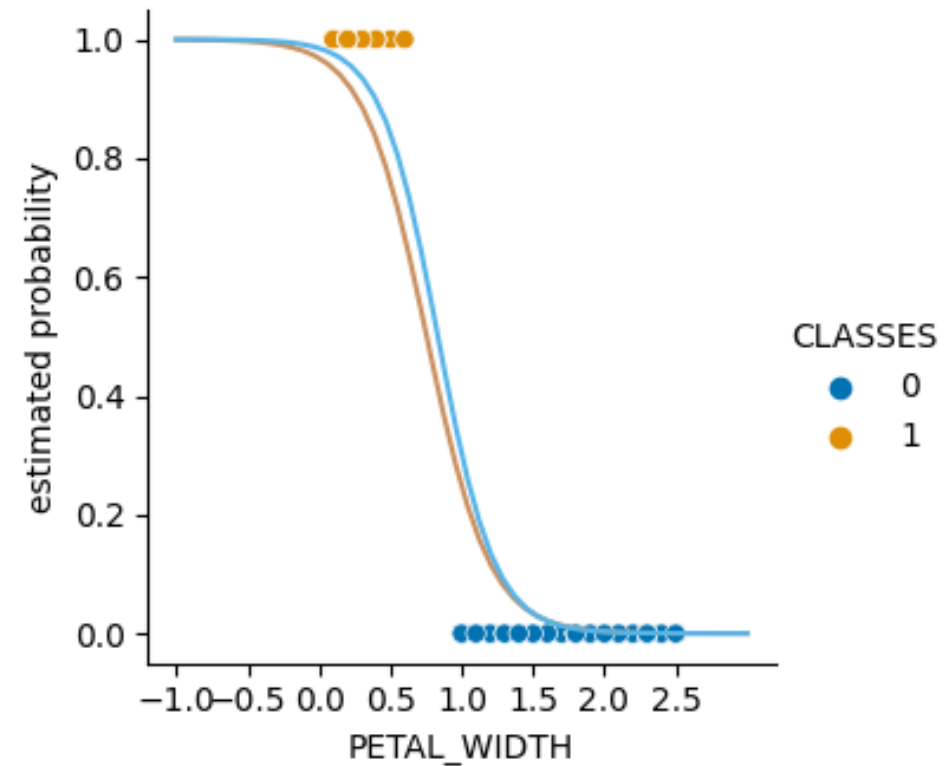
Decision boundary

$$\hat{p} = 0.5 = \frac{1}{1 + e^{-(3.42 - 4.53 * petal_width)}}$$



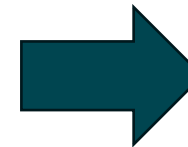
Decision boundary

- Decision boundary depends on the context / application
- Important to understand if datasets are unbalanced



Classification evaluation

		Predicted	
			
Observed		True Positives	False Negatives
		False Positives	True Negatives



Accuracy

Sensitivity / Recall

Precision / True positive value

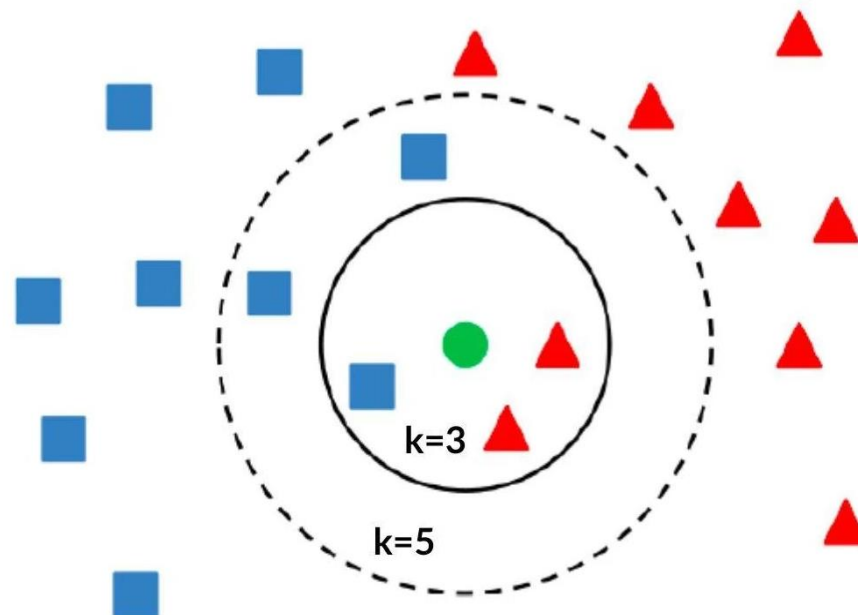
Specificity

F1 score

...

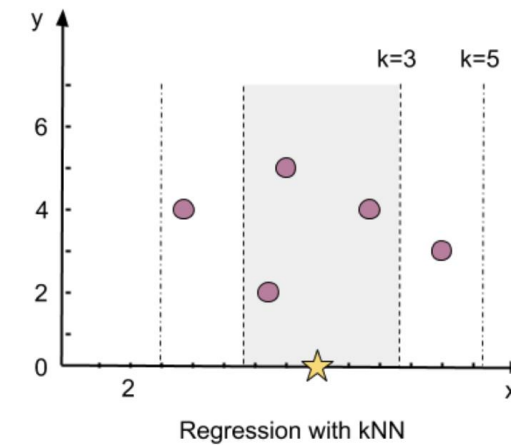
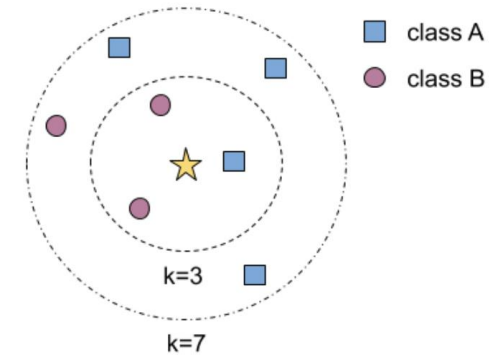
KNN

- Simple, non-parametric, lazy learning algorithm used for classification and regression tasks
- Classifies a data point based on how its neighbors are classified. In regression, it predicts the value based on the average of its neighbors.



KNN

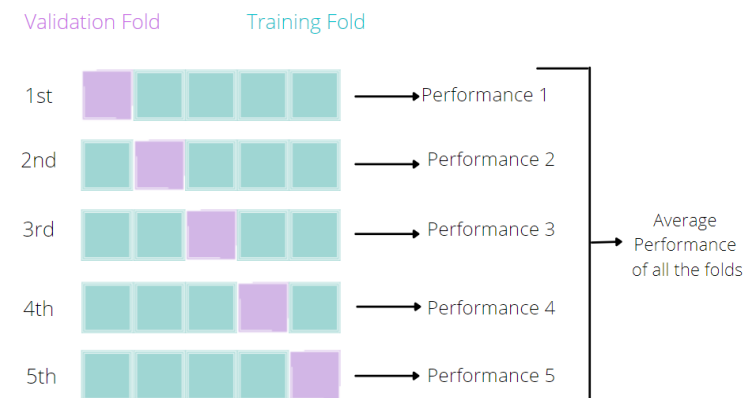
1. Choose the number of neighbors (K): The number of nearest neighbors to consider.
2. Calculate Distance: Use distance metrics (e.g., Euclidean, Manhattan) to find the K nearest neighbors.
3. Vote for Classification/Calculate for Regression:
 - **Classification:** The class most common among the neighbors is assigned to the data point.
 - **Regression:** The average of the neighbor values is taken as the prediction.



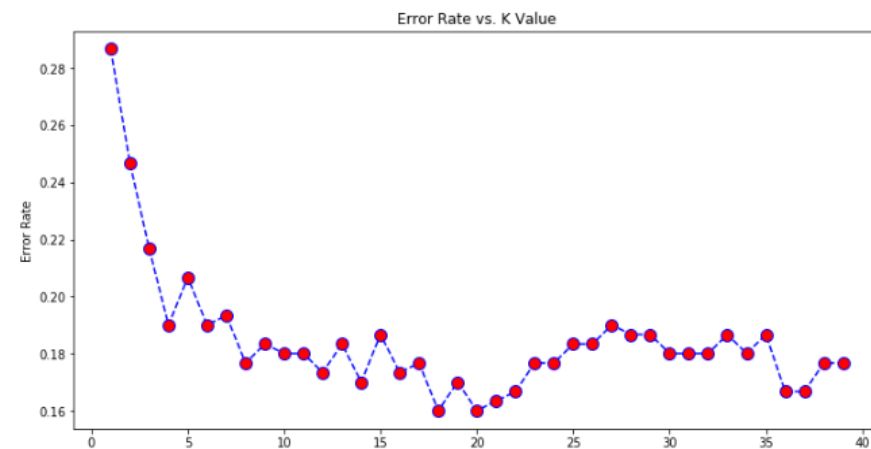
KNN

- How to choose K?

1. K-fold cross validation



2. Elbow method



KNN: recap

Pros

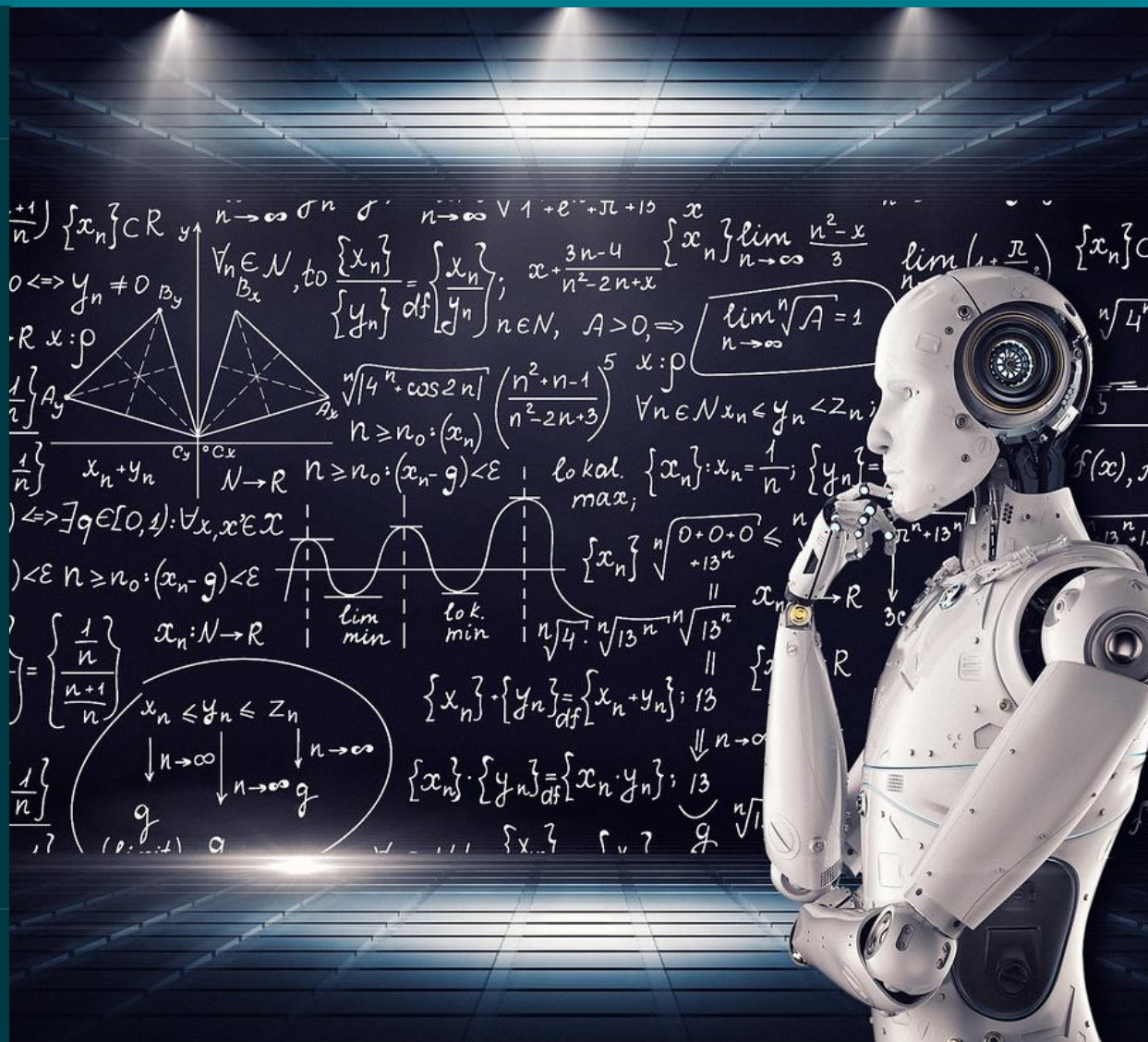
- Simple and easy to understand
- No assumptions about data distribution
- Effective with a small number of input variables.

Cons

- Computationally expensive, especially with large datasets
- Sensitive to the scale of data and irrelevant features
- Performance depends on the choice of K and distance metric
- Not really "learning"

Introduction to Machine Learning

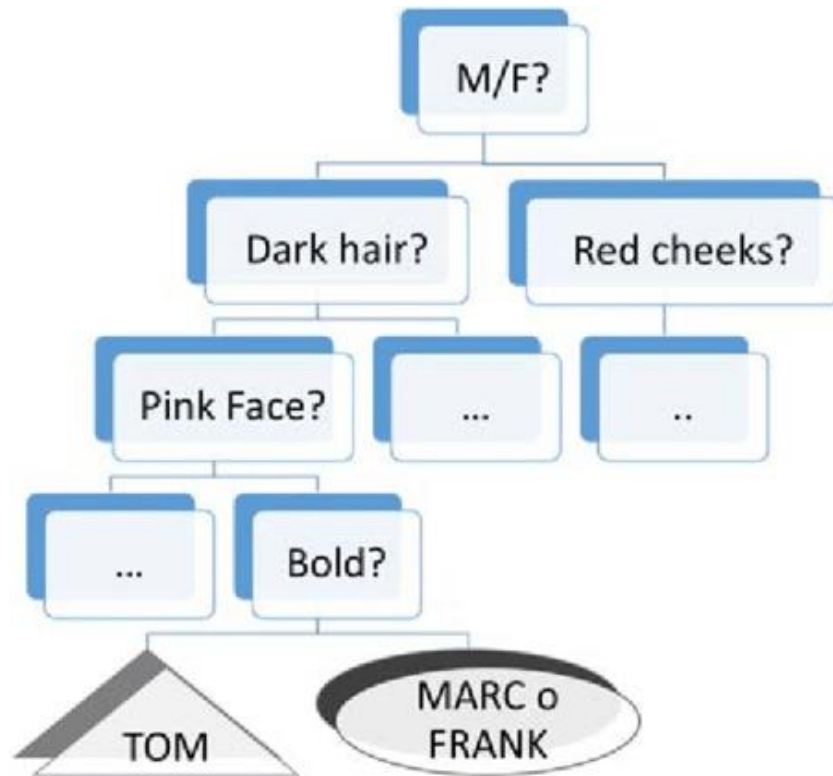
Tree-based methods



Talking about trees



Decision trees



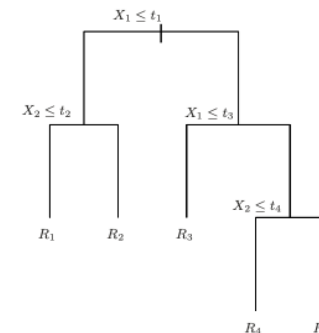
How do we find the number of predictor spaces?

- Goal: to minimize RSS or MSE (in regression)

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

- Steps:
 1. Start with all features and find j and s that minimize RSS
 2. Repeat the process within the result regions
 3. Continue until a stopping criteria is reached

$$\sum_{i: x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2, \dots$$



Avoiding overfitting: pruning the tree

- Finding subtrees
 - Experiment different subtrees and choose the one with the less error in CV
 - Cost complexity pruning:
 - Consider a sequence of trees to minimize error based on α terminal nodes

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

What about classification with decision trees?

- Same principle: instead of predicting the mean value we predict the **most commonly occurring class**
- Instead of minimizing RSS we use classification error rate

- Gini Index

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

- Entropy

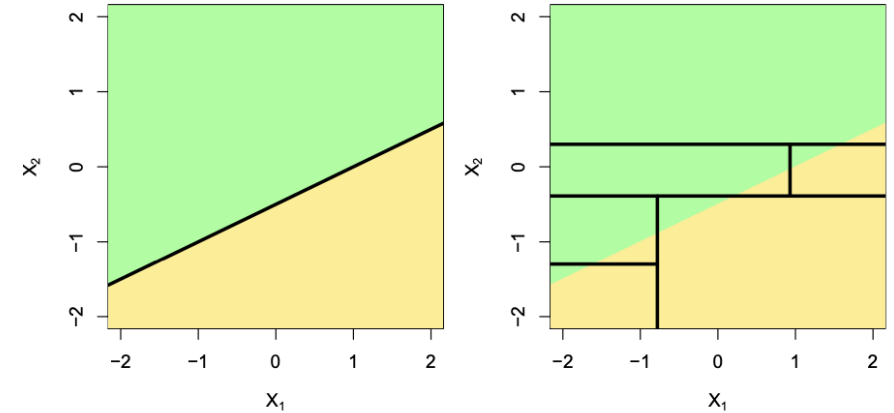
$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

Lower values indicate
"pure" nodes

Linear models vs decision trees

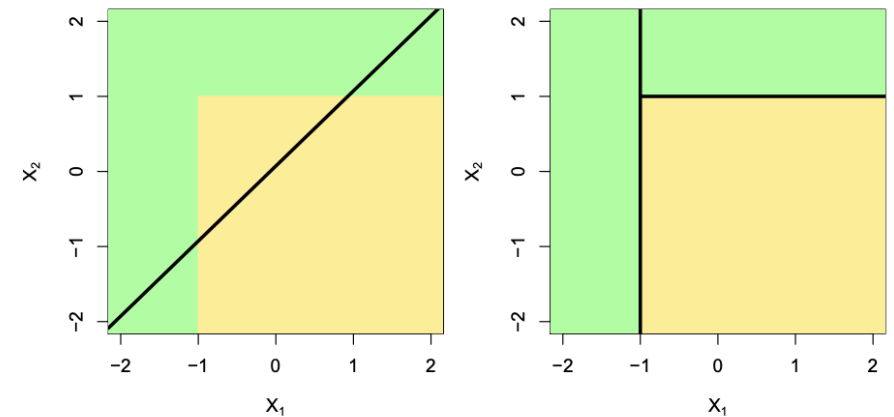
- Linear regression

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j,$$



- Decision tree

$$f(X) = \sum_{m=1}^M c_m \cdot 1_{(X \in R_m)}$$



Decision trees pros and cons

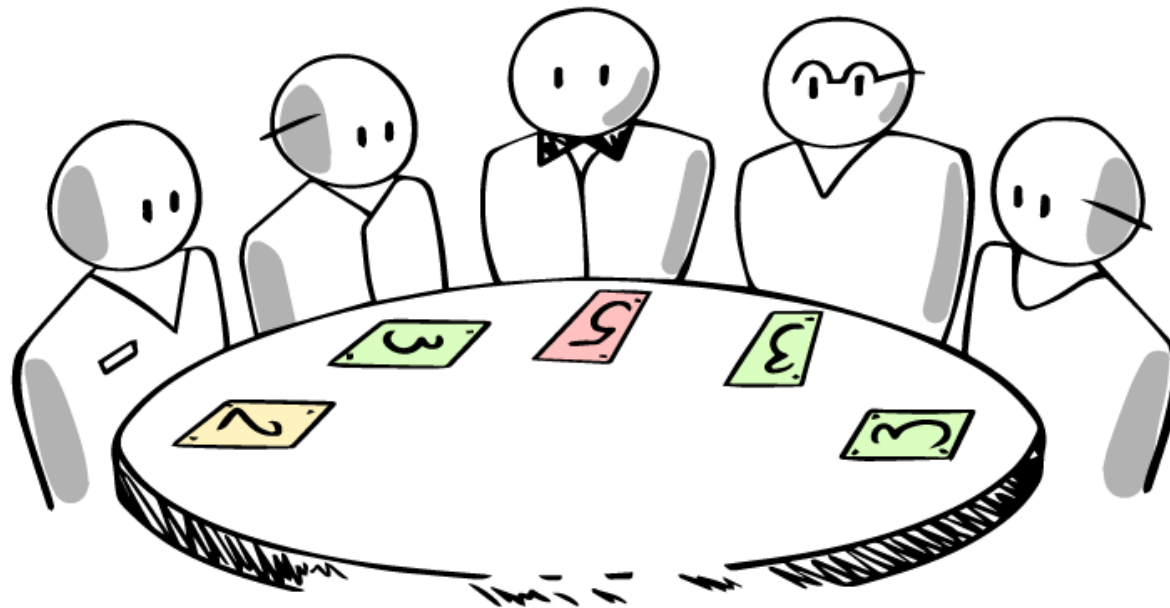
Pros

- Very easy to understand
- Easy to handle qualitative predictors
- No need to standardize data
- Easily displayed graphically

Cons

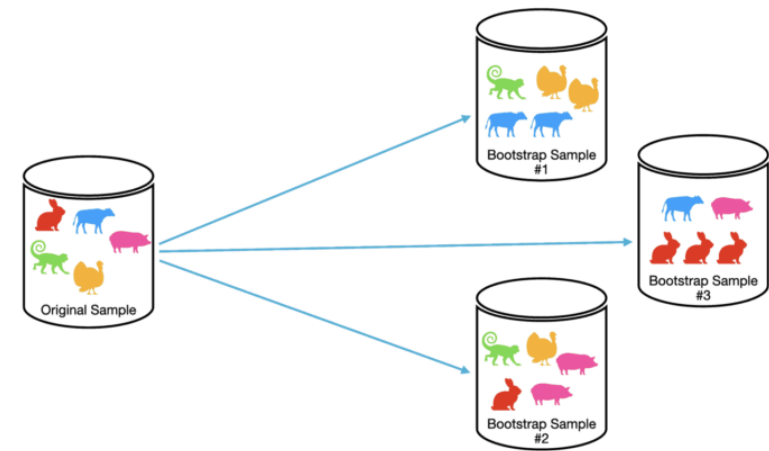
- Often not so accurate as other models
- Non-robust: a small change in data leads to different estimations
- Overfitting!!

From trees to forests



Bagging or Bootstrap aggregation

- Combining prediction of multiple models trained on different subsets of training data
- How it works:
 - Generate multiple sampled from original dataset
 - Each sample will have the same size
 - A model will be trained on each sample
 - Final model (ensemble) will take average predictions (regression) or majority vote (classification) from base learners
- Tradeoff for interpretability!



Regression

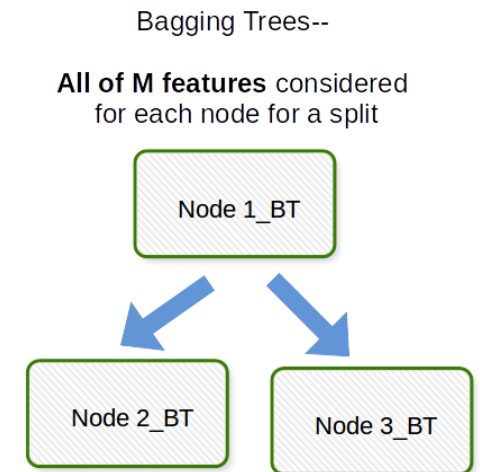
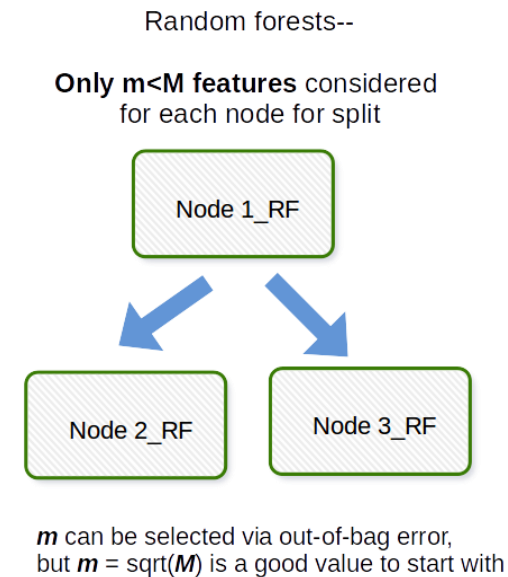
$$\hat{y} = \frac{1}{B} \sum_{i=1}^B M_i(x)$$

Classification

$$\hat{y} = \text{mode}(M_1(x), M_2(x), \dots, M_B(x))$$

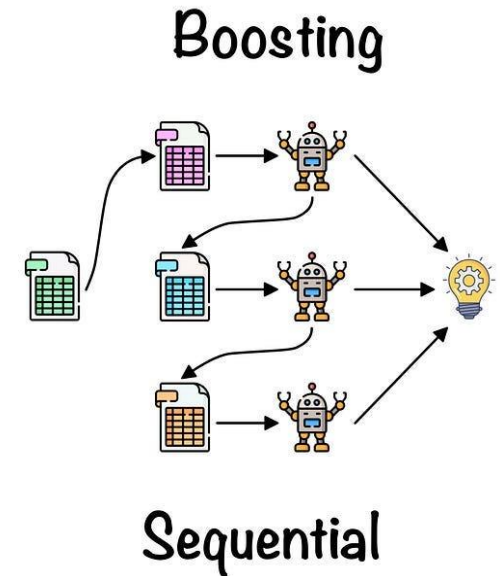
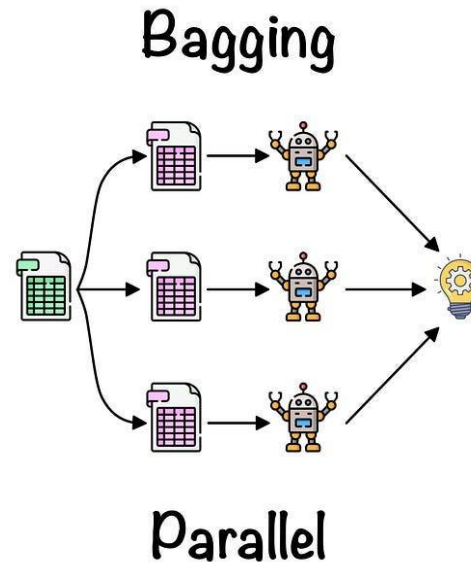
Random forests

- Also generate multiple bootstrap sampled from original training data but in addition, at each split only a **random subset of features** is considered for splitting
 - Typically square root number of total features



Boosting

- Like bagging but trees are grown **sequentially**
 - The input of each model is the residual error of the previous model
 - The goal is to improve where the first model did worse
- Each tree is fit on a modified version of the original dataset
- Leads to models that are good in different "parts" of the training data



Ensemble trees pros and cons

Pros

- Improved accuracy
- Reduced overfitting
- More robust to noise
- Deal well with mixed data types

Cons

- Computationally intensive
- Less interpretable
- Can have complex hyperparameter tuning (boosting machines)